Coursework 1 Task 2
Eaton Kralik
Game Engines

**1.1 Introduction.....1**
**1.2 Changes to the project.....1**
**1.3 Issues.....4**
**1.4 Transform vs Rigidbody.....4**
**1.5 Player movement evaluation.....4**
**1.6 Physics engine and collision detection.....5**
**1.7 Sources.....6**

# 1.1 Introduction

This portfolio task expands on the modular hotel corridor project, adding a movable player character, camera follow and physics-based collision events, with the use of assets provided by Unity's Essential Pathway template.

# 1.2 Changes to the project

The previous version of the scene used multiple copies of Unity's floor prefab to create the hallway, and this made the addition of box colliders time consuming. The current version of the modular hallway features only one floor prefab lengthened by the scale tool.

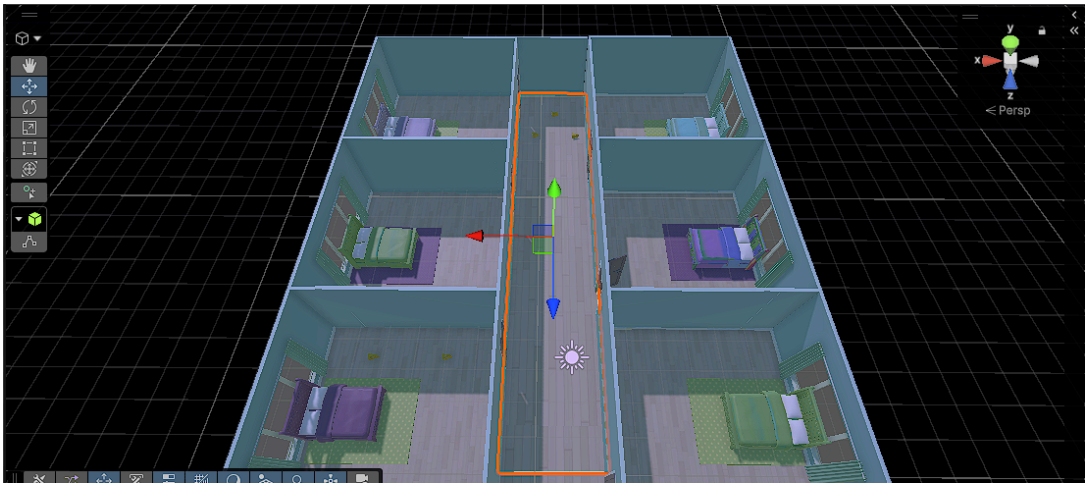



Figure 1: Singular floor prefab scaled to fit the length of one modular hallway half

Coursework 1 Task 2
Eaton Kralik
Game Engines

A movable player character has been added, using Unity's PlayerController script. The main camera has been assigned as a child of the player by dragging it under the player folder in the hierarchy.

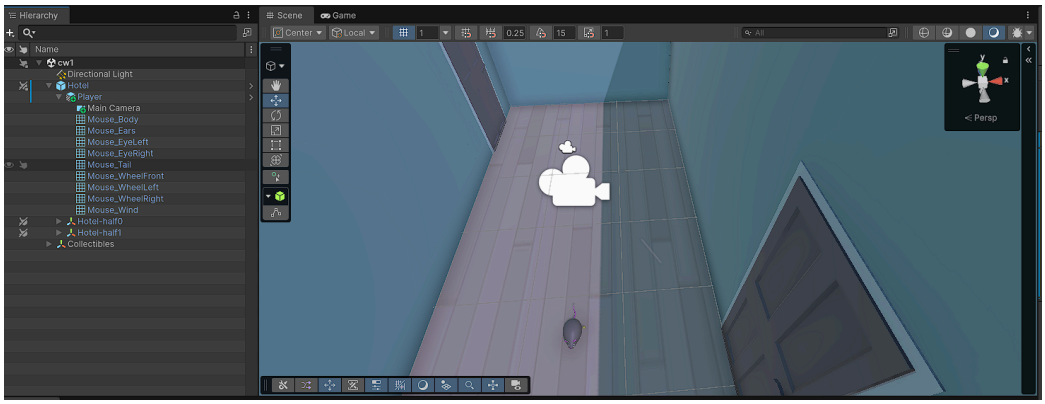



Figure 2: Hierarchy view of the Player and the main camera

```
C#
// Forward/backward
if (Keyboard.current.wKey.isPressed || Keyboard.current.upArrowKey.isPressed)
moveInput.y = 1f;
if (Keyboard.current.sKey.isPressed || Keyboard.current.downArrowKey.isPressed)
moveInput.y = -1f;

// Left/right (rotation)
if (Keyboard.current.aKey.isPressed || Keyboard.current.leftArrowKey.isPressed)
moveInput.x = -1f;
if (Keyboard.current.dKey.isPressed || Keyboard.current.rightArrowKey.isPressed)
moveInput.x = 1f;

// Move in facing direction
Vector3 movement = transform.forward * moveInput.y * speed * Time.fixedDeltaTime;
rb.MovePosition(rb.position + movement);

// Y-axis rotation (invert when going backwards)
float turnDirection = moveInput.x;
if (moveInput.y < 0)
    turnDirection = -turnDirection;

float turn = turnDirection * rotationSpeed * Time.fixedDeltaTime;
Quaternion turnRotation = Quaternion.Euler(0f, turn, 0f);
rb.MoveRotation(rb.rotation * turnRotation);
```

Figure 3: Code snippet from PlayerController script

Coursework 1 Task 2
Eaton Kralik
Game Engines

Closed doors are now openable, using a custom script placed in *Assets/_Unity Essentials/Scripts/My scripts*, that rotates the door on collision. Every door has been given a box collider and closed doors have been selected as Is Trigger. The script uses GetComponent to retrieve a component of a Collider type (Unity Technologies, 2026a) to disable the collider, making the door stay in an open position.

```C#
private void OnTriggerEnter(Collider other)
{
    transform.Rotate(0, degreeOfRotation, 0);
    this.GetComponent<Collider>().enabled = false;

}
```

Figure 4: Method OnTriggerEnter for the OpenDoor class

Rotating collectable objects have been added to the scene. They are not children of the Hotel parent to keep the custom hotel corridor prefab modular, however a collectible prefab containing its box collider and Collectible script can be found in the *Assets/_Unity Essentials/Prefabs/My prefabs* folder.

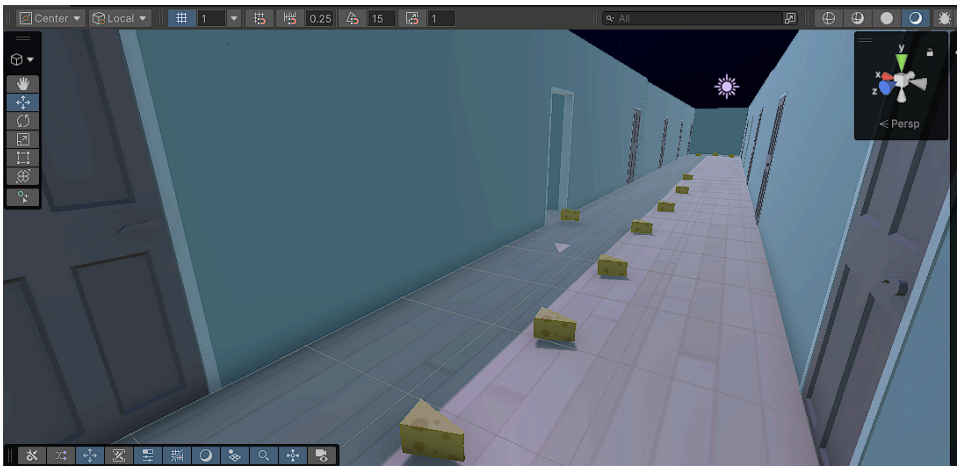



Figure 5: Cheese collectible placed in the scene




Figure 6: Cheese collectible in *Assets/_Unity Essentials/Prefabs/My prefabs*

Coursework 1 Task 2
Eaton Kralik
Game Engines

The project has been committed to Git and can be found at [kralik151/GameEngines_cw1: Coursework 1 for Game Engines at Edge Hill]

```
PS C:\Users\eytha\cw1> git log --graph
* commit b07e19ddec3d9cd1bdeb05af4f31892fc65dd07f (HEAD -> main, origin/main)
  Author: kralik151 <kralik151@uniba.sk>
  Date:   Fri Oct 30 09:39:49 2026 +0000

      CW1 Task 2
```

Figure 7: git log

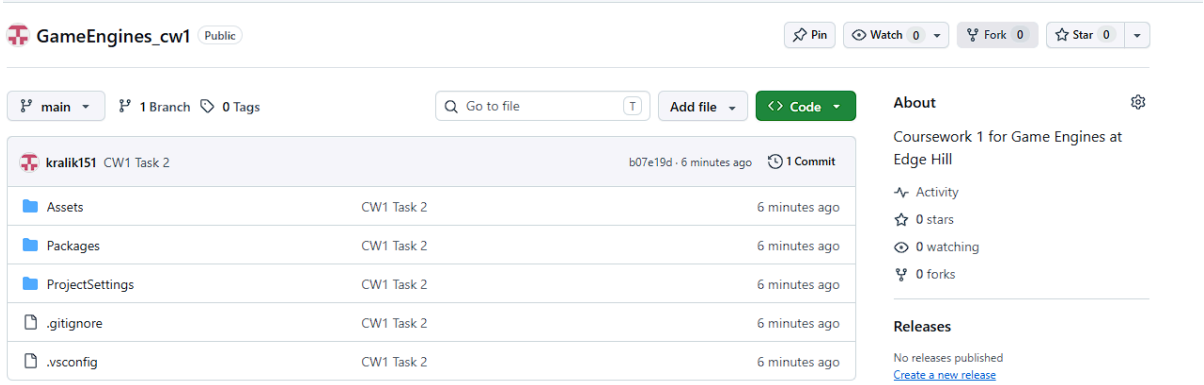



Figure 8: Github main

# 1.3 Issues

Currently it is difficult for the player to pass through the open doors. Occasionally the player character gets stuck, unable to enter a room at all or the character falls out of the map as a result of trying to enter. This could be because of the existing box collider in Unity Essential Pathway bedroom prefab, that was used for this project. A solution to this problem might be to remove the doorframe box collider entirely and replace it with multiple smaller colliders that don't intersect with the movable door. Due to the time constraints of this assignment, this issue will be resolved at a later date.

# 1.4 Transform vs Rigidbody

The transform based movement approach moves the object to specified coordinates, ignoring physics and making it possible for the object to pass through other objects or walls (Uhiyama, 2025). This project uses transform to rotate collectibles and doors, because they're simple movements and they don't need to interact with physics, or their environment.

Rigidbody movements are directly affected by game physics and each object has its mass and velocity, making collision and interaction with other objects possible (Uhiyama, 2025). Various physics based behavior can be configured in Unity in each Rigidbody item (Unity Technologies, 2026c). In this project, the rigidbody movement approach is used for the player character to provide collisions with walls and other objects in rooms.

# 1.5 Player movement evaluation

The PlayerController script used by Unity's Essential Pathway template uses a slightly outdated form of player movement, with the A and D buttons rotating the player instead of strafing left and right. This style of player control was used in older massively multiplayer online role-playing games (MMORPG). It can be found in games such as World of Warcraft Classic or Everquest, making the player control possible without the use of a computer mouse.

Even though this style of player control is not widely used in modern games and doesn't feature any acceleration, friction or momentum, it is still smooth and responsive, with the added benefit of the mouse being available for user interface (UI) interaction.

# 1.6 Physics engine and collision detection

The Unity physics engine simulates interaction between two objects, allowing them to respond to physical sources such as gravity or friction. Collision detection is an important aspect of physics simulation, preventing objects from intersecting.

There are multiple collision detection algorithms, including broad-phase and narrow-phase detection. Broad-phase lists all possible pairs of objects and discards all pairs that are too far away to collide, reducing the number of pairs necessary to check during collision. Narrow-phase then uses the list created by the broad-phase algorithm to perform precise intersection testing on each pair (Kockara et al., 2009).

Unity implements these algorithms in Collider and Rigidbody components. Collider components define physical boundaries of an object (Unity Technologies, 2026b), while Rigidbody components simulate physics based behavior such as movement, gravity, collision and joints (Unity Technologies, 2026c).

# 1.7 Sources

Unity Technologies (2026a) *GameObject.GetComponent*. Unity Editor 6000.6 Scripting API. Available at: https://docs.unity.com/en-us/engine/6000.6/script-reference/unityengine/gameobject/getcomponent (Accessed: 29 October 2026).

Unity Technologies (2026b) *Configure Rigidbody Colliders*. Unity User Manual 6000.6. Available at: https://docs.unity.com/en-us/engine/6000.6/manual/physics-section/physics-overview/rigidbody/rigidbody-configure-colliders (Accessed: 30 October 2026).

Unity Technologies (2026c) *Introduction to rigid body physics*. Unity User Manual 6000.6. Available at: https://docs.unity.com/en-us/engine/6000.6/manual/physics-section/physics-overview/rigidbody/rigidbodies-overview(Accessed: 30 October 2026).

Uhiyama (2025). *A complete guide to rigidbody, the heart of Unity's physics*. [online] UhiyamaLab. Available at: https://uhiyama-lab.com/en/notes/unity/unity-rigidbody-guide/ [Accessed 29 Oct. 2026].

Kockara, S., Halic, T., Bayrak, C., Iqbal, K. and Rowe, R.A. (2009). Contact detection algorithms. *Journal of Computers*, [online] 4(10). doi:10.4304/jcp.4.10.1053-1063.